

# Aula 10 — Máquinas de Vetores de Suporte (SVM)

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §8.2 e §4.6; ISLP cap. 9

## O que você vai aprender

Ao final desta aula você deve ser capaz de:

- definir hiperplano separador e o **classificador de margem máxima**;
- identificar os **vetores de suporte** e o papel da margem;
- generalizar para dados não separáveis com o **classificador de vetor de suporte** (*soft margin*) e o hiperparâmetro  $C$ ;
- explicar o **truque do kernel** a partir da forma dual;
- usar *kernels* (polinomial, gaussiano/RBF) para fronteiras não lineares e entender  $\gamma$ ;
- reconhecer a SVM como minimização da *perda* hinge regularizada e sua relação com a regressão logística; tratar o caso multiclasse.

Mudança de perspectiva em relação à Aula 08. Lá, todo método passava por modelar  $\mathbb{P}(Y | \mathbf{x})$  e depois aplicar o Teorema de Bayes. A SVM ignora probabilidades e ataca a fronteira *geometricamente*, procurando a separação mais folgada possível. É um dos métodos mais elegantes do curso — liga geometria, otimização e os RKHS da Aula 04. Comece pela pergunta:

*se há infinitos hiperplanos que separam as classes, qual é o melhor?*

## 1 Hiperplanos e o classificador de margem máxima

Codifique as classes como  $y \in \{-1, +1\}$ . Um **hiperplano** em  $\mathbb{R}^d$  é o conjunto  $\{\mathbf{x} : f(\mathbf{x}) = 0\}$  com  $f(\mathbf{x}) = \beta_0 + \beta^\top \mathbf{x}$ . Ele separa o espaço em dois lados conforme o sinal de  $f$ . Dizemos que os dados são *linearmente separáveis* se existe  $f$  tal que

$$y_i f(\mathbf{X}_i) > 0 \quad \text{para todo } i. \quad (1)$$

A magnitude  $|f(\mathbf{x})|$  (com  $\|\beta\| = 1$ ) é a *distância* de  $\mathbf{x}$  ao hiperplano.

**Ideia central:** por que a codificação  $\pm 1$

Não é capricho de notação. Com  $y \in \{-1, +1\}$ , o produto  $y_i f(\mathbf{X}_i)$  é positivo exatamente quando a classificação está certa, e seu *valor* mede o quanto ela está certa. Essa única quantidade — a **margem** da observação  $i$  — vai organizar a aula inteira, inclusive a função de perda da Seção 4.

**Definição 1.1** (Classificador de margem máxima). Procura-se o hiperplano que maximiza a margem  $M$ :

$$\max_{\beta_0, \beta} M \quad \text{s.a.} \quad \sum_{j=1}^d \beta_j^2 = 1, \quad y_i f(\mathbf{X}_i) \geq M \quad \forall i.$$

A restrição  $\|\beta\| = 1$  torna os hiperplanos comparáveis (assim  $y_i f(\mathbf{X}_i)$  é a distância com sinal). As observações que *atingem* a margem ( $y_i f(\mathbf{X}_i) = M$ ) são os **vetores de suporte**.

**Atenção**

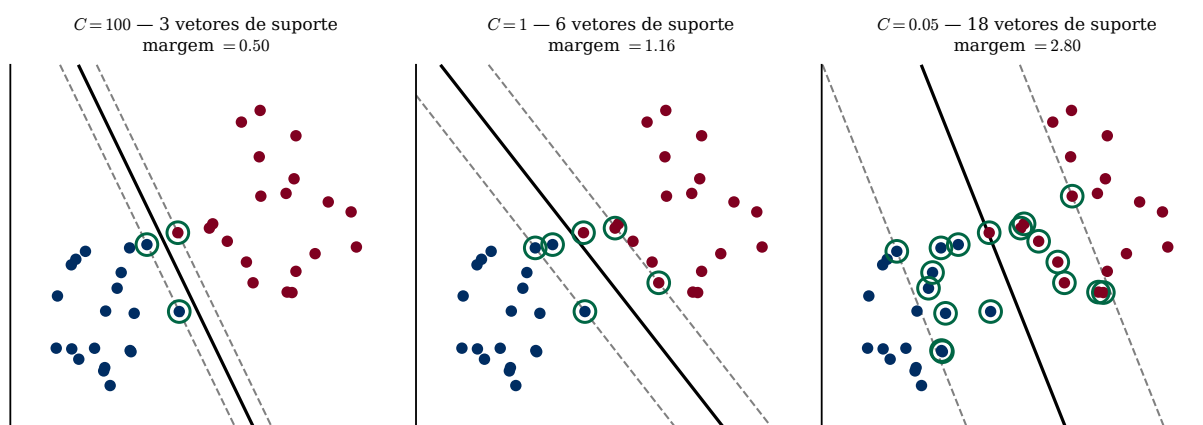
Apenas os vetores de suporte determinam o hiperplano: mover um ponto longe da margem não muda *nada*. Isso é bom e ruim ao mesmo tempo. Bom: o modelo é econômico, definido por um punhado de observações. Ruim: a formulação *exige* separabilidade perfeita e é **muito sensível** — mover um único vetor de suporte pode girar a fronteira inteira. Precisamos relaxá-la.

## 2 O classificador de vetor de suporte (*soft margin*)

Para lidar com dados não separáveis (e ganhar robustez), permitimos que algumas observações violem a margem, introduzindo *folgas*  $e_i \geq 0$ :

$$\max_{\beta_0, \beta, e} M \quad \text{s.a.} \quad \|\beta\| = 1, \quad y_i f(\mathbf{X}_i) \geq M(1 - e_i) \quad \forall i, \quad e_i \geq 0, \quad \sum_{i=1}^n e_i \leq C. \quad (2)$$

A folga  $e_i$  mede o quanto a  $i$ -ésima observação viola a margem:  $e_i > 0$  significa dentro da margem;  $e_i > 1$  significa do *lado errado* do hiperplano (erro de classificação).



**Figura 1:** O mesmo conjunto de 40 pontos com três orçamentos de violação. A linha cheia é a fronteira; as tracejadas delimitam a margem; os pontos circutados em verde são os **vetores de suporte**. Aumentar a tolerância a violações alarga a margem e traz mais pontos para dentro dela: de 3 vetores de suporte e margem 0,50 até 18 vetores de suporte e margem 2,80. Quanto mais pontos sustentam a solução, menos ela depende de cada um — daí a robustez. **Atenção ao rótulo:** este é o  $C$  do `scikit-learn`, cuja convenção é a inversa da de (2).

**Ideia central**

$C$  é o **tuning parameter**, escolhido por validação cruzada (Aula 03). Na convenção de (2) — a do [AME] e do [ISLP] —  $C$  é o *orçamento de violações*:

- $C$  *grande*: tolera muitas violações  $\Rightarrow$  margem larga, mais vetores de suporte, modelo mais *rígido* (alto viés, baixa variância);
- $C$  *pequeno*: poucas violações  $\Rightarrow$  margem estreita, modelo mais *flexível* (baixo viés, alta variância).

É o balanço viés-variância da Aula 01 outra vez.

**Atenção: a inversão que já derrubou muita gente**

No `scikit-learn`, o  $C$  é o *peso da penalidade* sobre as violações, não o orçamento delas. Portanto tudo se inverte:  $C$  grande penaliza mais, tolera menos, e dá margem *estreita*. É o que a Figura 1 mostra. Ao ler qualquer texto sobre SVM, confira qual convenção está em uso antes de interpretar “ $C$  grande”.

### 3 O truque do *kernel*

A chave que torna a SVM poderosa: a solução depende dos dados *apenas através de produtos internos*. Pode-se mostrar que a função ótima se escreve

$$f(\mathbf{x}) = \beta_0 + \sum_{k=1}^n \alpha_k \langle \mathbf{x}, \mathbf{X}_k \rangle, \quad \langle \mathbf{x}, \mathbf{X}_k \rangle = \sum_{j=1}^d x_j x_{k,j}, \quad (3)$$

e que o cálculo dos  $\alpha_k$  também só requer os produtos internos  $\langle \mathbf{X}_i, \mathbf{X}_k \rangle$  entre as observações. Crucialmente,  $\alpha_k = 0$  para todos os pontos que *não* são vetores de suporte; logo

$$f(\mathbf{x}) = \beta_0 + \sum_{k \in S} \alpha_k \langle \mathbf{x}, \mathbf{X}_k \rangle,$$

onde  $S$  é o conjunto de vetores de suporte.

**Ideia central**

Como tudo depende só de produtos internos, podemos **substituir**  $\langle \mathbf{x}, \mathbf{X}_k \rangle$  por um *kernel*  $K(\mathbf{x}, \mathbf{X}_k)$  — que corresponde a um produto interno num espaço de *features* de dimensão muito maior (ou infinita), sem *nunca* computar esse espaço explicitamente. Isso é o **truque do kernel**: fronteiras não lineares no espaço original a custo de fronteiras lineares num espaço ampliado.

**Exemplo 3.1** (o truque em duas dimensões). Tome  $K(\mathbf{x}, \mathbf{x}') = (\langle \mathbf{x}, \mathbf{x}' \rangle)^2$  em  $\mathbb{R}^2$ . Expandindo,

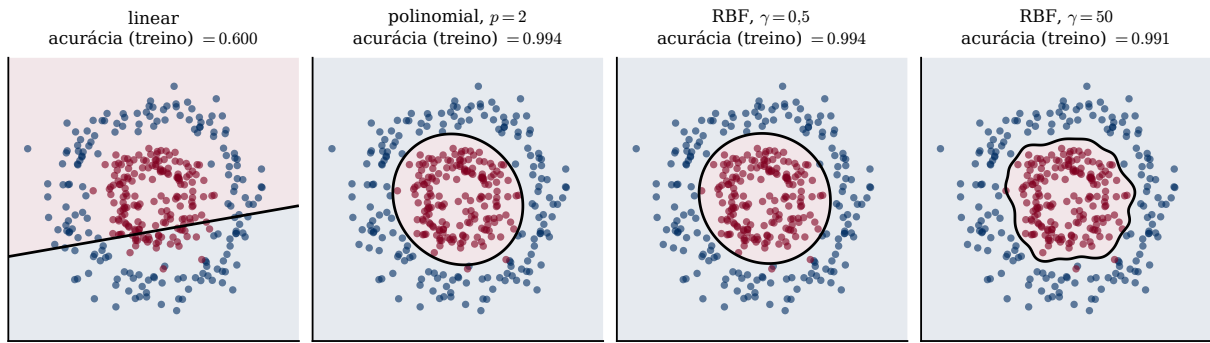
$$(x_1 x'_1 + x_2 x'_2)^2 = (x_1^2)(x_1'^2) + 2(x_1 x_2)(x_1' x_2') + (x_2^2)(x_2'^2) = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle,$$

com  $\phi(\mathbf{x}) = (x_1^2, \sqrt{2} x_1 x_2, x_2^2)$ . Ou seja: calcular *um* produto e elevá-lo ao quadrado equivale a mapear os pontos para  $\mathbb{R}^3$  e fazer o produto interno lá. Com o kernel polinomial de grau  $p$  em  $\mathbb{R}^d$  o espaço tem dimensão  $\binom{d+p}{p}$ , e com o RBF ele é *de dimensão infinita* — e o custo continua sendo uma conta com  $d$  termos. É esse o negócio.

*Kernels* de Mercer comuns (cf. Aula 04, §4.6):

| Kernel          | $K(\mathbf{x}, \mathbf{x}')$                      |
|-----------------|---------------------------------------------------|
| Linear          | $\langle \mathbf{x}, \mathbf{x}' \rangle$         |
| Polinomial      | $(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^p$ |
| Gaussiano (RBF) | $\exp(-\gamma \ \mathbf{x} - \mathbf{x}'\ ^2)$    |

No kernel gaussiano,  $\gamma > 0$  é um *tuning parameter*:  $\gamma$  grande  $\Rightarrow$  influência local, fronteiras muito flexíveis (risco de superajuste);  $\gamma$  pequeno  $\Rightarrow$  fronteiras suaves. Escolhem-se  $C$  e  $\gamma$  juntos por validação cruzada.



**Figura 2:** Duas classes dispostas em anéis concêntricos — o exemplo canônico do que uma reta não resolve. O kernel linear acerta 60%, pouco acima do palpite. O polinomial de grau 2 encontra a circunferência exata e sobe para 99,4%, e o RBF com  $\gamma = 0,5$  faz o mesmo. Já o RBF com  $\gamma = 50$  produz uma fronteira rendilhada que persegue observações individuais:  $\gamma$  é um botão de flexibilidade e, como todos os outros do curso, superajusta se você o girar demais.

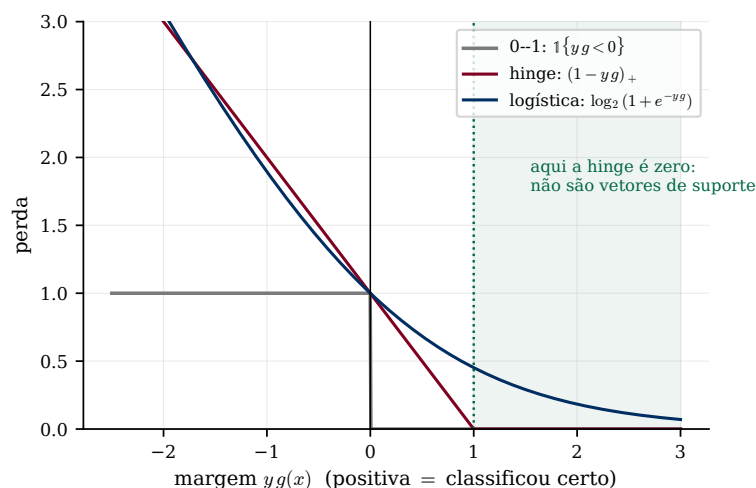
## 4 SVM como perda *hinge* regularizada

Há uma leitura que conecta a SVM a tudo o que vimos. Dado um kernel de Mercer  $K$  com RKHS  $\mathcal{H}_K$  (Aula 04), o classificador da SVM minimiza

$$\min_{g \in \mathcal{H}_K} \sum_{k=1}^n \underbrace{(1 - y_k g(\mathbf{X}_k))_+}_{\text{perda hinge}} + \lambda \|g\|_{\mathcal{H}_K}^2, \quad (4)$$

onde  $(u)_+ = \max(u, 0)$ . Ou seja, a SVM é *ajuste de perda + regularização* (Aula 02), com a **perda *hinge***  $L(y, g) = (1 - yg)_+$ .

**Observação 4.1.** A perda *hinge* é **zero** quando  $yg(\mathbf{x}) \geq 1$ , isto é, quando o ponto está do lado certo e *folgadoamente* longe da margem. Só os pontos próximos ou violando a margem contribuem — e são exatamente os *vetores de suporte*. No caso separável, (4) reencontra a margem máxima  $(1/\|g\|)$ .



**Figura 3:** As três perdas como função da margem  $yg(x)$ . A 0-1 é o que gostaríamos de minimizar, mas ela é descontínua e constante por partes — otimizar sobre ela é NP-difícil. A *hinge* e a logística são substitutas convexas: ambas majoram a 0-1 e são tratáveis. A diferença decisiva está na faixa sombreada: a *hinge* **zero** a partir da margem 1, e todos os pontos ali param de influenciar a solução — é daí que vem a esparsidade em vetores de suporte. A logística nunca chega a zero, e por isso *todo* ponto pesa um pouco.

## 4.1 Relação com a regressão logística

A SVM (*hinge*) e a regressão logística (perda logística  $\log(1 + e^{-y\theta})$ ) são ambas “perda de margem + regularização  $\ell_2$ ”:

- a *hinge* é exatamente zero longe da margem  $\Rightarrow$  solução *esparsa* em vetores de suporte; a logística nunca é zero;
- a logística entrega **probabilidades** calibradas (Aula 09); a SVM entrega só a fronteira. As “probabilidades” do SVC vêm de um ajuste *a posteriori* (escala de Platt), não do método.

Regra prática: classes bem separadas  $\rightarrow$  SVM costuma ir muito bem; quando se quer  $\mathbb{P}(Y | \mathbf{x})$ , prefira a logística.

## 5 Mais de duas classes

A SVM é intrinsecamente binária — não há como escrever “ $y_i f(\mathbf{X}_i) > 0$ ” com três classes. Para  $|\mathcal{C}| > 2$ , contorna-se com vários classificadores binários:

### Um-contra-um (OvO)

treina uma SVM para cada par de classes ( $\binom{K}{2}$  classificadores) e classifica por votação. Cada ajuste usa só os dados de duas classes, então são muitos problemas pequenos. Padrão do `scikit-learn`.

### Um-contra-todos (OvA)

treina  $K$  SVMs (cada classe vs. o resto) e escolhe a de maior  $f(\mathbf{x})$ . São  $K$  problemas grandes, e cada um deles fica desbalanceado — com a ressalva da Aula 09.

## 6 Prática em Python

```

1 from sklearn.svm import SVC
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4 from sklearn.model_selection import GridSearchCV
5
6 # SVM com kernel RBF; padronizar e ESSENCIAL (metodo baseado em distancia)
7 pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
8 grade = {"svc__C": [0.1, 1, 10, 100],
9          "svc__gamma": [0.001, 0.01, 0.1, 1]}
10 busca = GridSearchCV(pipe, grade, cv=5, scoring="accuracy")
11 busca.fit(X_tr, y_tr)
12 print("melhores parametros:", busca.best_params_)
13
14 # Para probabilidades: SVC(probability=True) (usa Platt scaling, mais lento)

```

### Atenção

Três observações práticas. (i) SVM com RBF é **sensível à escala** — a distância  $\|\mathbf{x} - \mathbf{x}'\|^2$  do kernel soma todas as coordenadas, então *sempre* padronize dentro do *pipeline* (Aula 07). (ii)  $C$  e  $\gamma$  interagem, e por isso a busca é numa *grade bidimensional*, não em dois passos separados. (iii) O custo de treino do SVC cresce entre  $O(n^2)$  e  $O(n^3)$ : com centenas de milhares de observações ele fica inviável, e a saída é o `LinearSVC` ou aproximações de kernel (`Nystroem`).

Os notebooks **Aula prática 10** e **Exemplo - SVM em dados artificiais** ilustram o efeito de  $C$ ,  $\gamma$  e da escolha do kernel sobre a fronteira de decisão.

## Resumo

- **Margem máxima:** hiperplano separador de maior folga; definido pelos *vetores de suporte*; exige separabilidade e é frágil.
- **Soft margin** (2): folgas  $e_i$  e orçamento  $C$  controlam o balanço viés-variância (Fig. 1). Cuidado com a convenção invertida de  $C$  no `scikit-learn`.
- **Truque do kernel:** a solução só depende de produtos internos (3); trocá-los por  $K$  dá fronteiras não lineares (Fig. 2), e  $\gamma$  do RBF é mais um botão de flexibilidade.
- SVM = **perda hinge** + regularização (4) (RKHS, Aula 04); a hinge zera longe da margem, e é daí que vem a esparsidade (Fig. 3).
- Relação com a logística (perda de margem +  $\ell_2$ ); SVM não dá probabilidades de imediato.
- Multiclasse por OvO/OvA; *padronize* sempre.

**Leitura recomendada.** [AME] §8.2 (SVM) e §4.6 (RKHS e o truque do kernel, onde o mesmo maquinário aparece em regressão). [ISLP] Capítulo 9 inteiro: §9.1 (margem máxima), §9.2 (vetor de suporte e o papel de  $C$  — a Figura 9.7 deles é a nossa Figura 1), §9.3 (kernels, com a versão deles da nossa Figura 2), §9.4 (multiclasse) e §9.5 (relação com a logística, de onde vem a nossa Figura 3).

**Para praticar.** `recursos/listas/Lista de exercícios 08.pdf`.